

1. Định nghĩa API

- Theo JJ Geewax:

- API (Application Programming Interface) là giao diện định nghĩa cách các hệ thống máy tính tương tác với nhau.
- Đối với Web API, đây là các giao diện được thiết kế để kết nối qua mạng, giúp che giấu đi sự phức tạp về tính toán hoặc cấu trúc cài đặt bên dưới.
- API đóng vai trò quan trọng trong việc tự động hóa và cho phép lắp ráp (composition) các chức năng phần mềm như những khối Lego.

- Theo James Higginbotham:

- API không chỉ là công nghệ mà còn là phương tiện giao tiếp (communication) giữa hệ thống với hệ thống, với lập trình viên và với thị trường.
- Web API được thiết kế theo tư duy sản phẩm (product thinking), cung cấp khả năng kinh doanh (business capabilities) cho tổ chức và đặc biệt chú trọng vào trải nghiệm của nhà phát triển (developer experience - DX).

2. Định nghĩa Resource (Tài nguyên)

- Theo JJ Geewax:

- Tài nguyên là những khái niệm cốt lõi mà API quản lý (các "sự vật" trong hệ thống).
- Các API hướng tài nguyên (Resource-oriented APIs) cung cấp một bộ các phương thức chuẩn (như Create, Get, List, Update, Delete) để tương tác với các tài nguyên này, giúp API dễ học và dễ nhớ hơn so với các hệ thống thuần RPC (Remote Procedure Call).

- Theo James Higginbotham:

- Tài nguyên là đại diện kỹ thuật số của một khái niệm (một thực thể, tập hợp thực thể, hoặc thậm chí là quy trình kinh doanh).
- **Tài nguyên không phải là Data Models:** API không nên phản chiếu trực tiếp lược đồ cơ sở dữ liệu để tránh làm API trở nên dễ vỡ khi dữ liệu thay đổi.
- **Tài nguyên không phải là Object/Domain Models:** API nên tập trung vào thông điệp (messages) trao đổi giữa hệ thống và người dùng thay vì bám chặt

vào các cấu trúc đối tượng nội bộ.

3. Đặc điểm của một API tốt

- Theo JJ Geewax (4 yếu tố chính tạo nên API tốt):
 - **Operational (Hoạt động tốt):** API phải thực sự thực hiện đúng các chức năng mà người dùng cần và đáp ứng các yêu cầu phi chức năng (độ trễ, tính chính xác).
 - **Expressive (Tính biểu đạt cao):** Cho phép người dùng thể hiện rõ ràng và đơn giản điều họ muốn thực hiện.
 - **Simple (Đơn giản):** Thiết kế sao cho "những trường hợp sử dụng phổ biến nhất thì cực kỳ dễ dàng, và những trường hợp phức tạp thì vẫn có thể thực hiện được".
 - **Predictable (Có thể dự đoán được):** API dựa trên các mẫu lặp lại (patterns) đồng nhất, nhất quán trong cách đặt tên và cấu trúc để không gây bất ngờ cho lập trình viên.
- Theo James Higginbotham (5 Nguyên tắc thiết kế Web API):
 - **Collaborative (Hợp tác):** API không bao giờ nên được thiết kế cô lập mà cần sự cộng tác của nhiều bên.
 - **Outcome-based (Dựa trên kết quả):** Thiết kế phải bắt đầu bằng việc tập trung vào giá trị (outcomes) mang lại cho hệ thống/người dùng.
 - **Match the need (Phù hợp nhu cầu):** Chọn các yếu tố thiết kế, giao thức phù hợp (REST, GraphQL, gRPC...) dựa trên nhu cầu thực tế chứ không gò ép.
 - **Documentation as UI (Tài liệu là giao diện):** Tài liệu API là giao diện người dùng quan trọng nhất cho lập trình viên, cần được ưu tiên hàng đầu.
 - **APIs are forever (API là mãi mãi):** API rất khó thay đổi sau khi phát hành, nên cần thiết kế cẩn thận và có chiến lược nâng cấp (versioning) tiến hóa theo thời gian.

4. So sánh với 5 đặc điểm cốt lõi của API hiện đại (Trong Slide 05)

**(Ghi chú: 5 đặc điểm trên Slide 05 là: 1. Có hợp đồng rõ ràng (Contract-first); 2. Ẩn cài đặt (Abstraction); 3. Có thể tái sử dụng (Reusability); 4. Độc lập với ngôn ngữ &*

*nền tảng (Language-agnostic); 5. Có thể versioning & quan sát được (Versioned & Observable))**

- **Điểm tương đồng:**
 - **Ẩn cài đặt (Abstraction):** Rất đồng nhất với cả hai tác giả. Geewax coi API là lớp vỏ che giấu sự phức tạp tính toán, còn Higginbotham nhấn mạnh tính Encapsulation (đóng gói) và khẳng định "Resource không phải là Data Model".
 - **Có thể tái sử dụng (Reusability):** Phù hợp với quan điểm "Composition" (lắp ráp) của Geewax và "Product Thinking" (tư duy sản phẩm phục vụ nhiều đối tượng) của Higginbotham.
 - **Có hợp đồng rõ ràng (Contract-first):** Phản ánh qua việc Higginbotham coi API Design là quá trình "Communication" và coi tài liệu là giao diện người dùng chính (Documentation as UI). Geewax cũng nhắc đến tính "Predictable" (có thể dự đoán được) thông qua hợp đồng và các quy ước rõ ràng.
 - **Có thể versioning:** Tương đương với nguyên tắc "APIs are forever" của Higginbotham, đòi hỏi thiết kế và quản lý sự thay đổi cẩn thận.
- **Điểm mới mà Geewax / Higginbotham bổ sung thêm:**
 - **Tập trung vào Developer Experience (Trải nghiệm của lập trình viên):** Slide 05 tập trung nhiều vào kỹ thuật và kiến trúc, trong khi Geewax bổ sung thêm khía cạnh về khả năng sử dụng: API cần phải **Đơn giản (Simple)**, **Có khả năng biểu đạt (Expressive)**, và **Dễ dự đoán (Predictable)**. Higginbotham cũng lấy "Developer Experience" làm một trong 3 trụ cột của Web API Design.
 - **Tính hướng đích và thực tiễn kinh doanh:** Higginbotham bổ sung nguyên tắc **Outcome-based focus** (tập trung vào kết quả) và **Business Capabilities** (Khả năng kinh doanh). API phải mang lại giá trị thiết thực và "Match the need" (giải quyết đúng vấn đề) chứ không chỉ đáp ứng đúng chuẩn kiến trúc.
 - **Tính cộng tác trong thiết kế:** Higginbotham bổ sung quá trình thiết kế API (Collaborative design), nhấn mạnh rằng việc tạo ra một API tốt là kết quả của sự hợp tác thay vì thiết kế đơn độc từ một nhóm nhỏ.